

Assignment: Real-Time Android Device in the Browser

Take-home assignment

Deadline	72 hours from the moment you receive this assignment
What you build	A live Android device streamed to a web page, with touch and keyboard input
What you submit	Public Git repository (backend and frontend), deployed link, demo video, write-up, AI record
Constraints	Core solution built with free or open-source software. Hosting can be any provider, free or paid.
Questions	Reply to the assignment email or write to rohit.ranjan@myhealthtick.com

Overview

Build a web application that shows a live, interactive Android device inside a web page. A user opens the page, sees the device screen update in real time, and operates it with mouse and keyboard as if it were a physical phone.

This is an open-ended problem. There is no single correct architecture, and the pieces are not handed to you. We expect you to research, try things, hit walls, break your setup a few times, and work your way to something that runs. We care as much about how you get there as about the result, so keep notes on what you tried and what failed.

You choose the tools, libraries, and approach. The device and streaming solution itself must be built with free or open-source software. Do not use any paid product that provides the core functionality for you, such as a hosted device-streaming or remote-device service, a paid emulator or device-farm platform, or a commercial streaming SDK. Justify your choices in the write-up.

This restriction applies only to how you solve the problem. Hosting is separate: you are free to use any cloud provider or managed infrastructure to deploy and run your solution, including paid plans, and you do not need to keep hosting costs at zero.

Core requirements

1. The page displays the live screen of a running Android device, updating continuously without a manual refresh.
2. The user can tap, swipe, scroll, and type from the browser. Input must land at the correct position on the device screen regardless of browser window size.
3. The experience feels responsive. Measure the delay between an action and the visible screen update, describe how you measured it, and report the numbers.
4. The project runs on a single machine from documented setup steps that another person can follow.
5. The app is deployed and reachable through a public link that we can open and test ourselves (see Deployment).

Bonus requirements

Each bonus is judged on depth and correctness, not on whether it merely exists.

1. Dedicated isolated instance per user. Two users opening the page at the same time each get their own device. Actions, files, settings, and installed state must not leak between them.
2. Instance on demand. A device is created when the user requests a session and released when the session ends or goes idle. Nothing is reserved per user in advance, and abandoned sessions must not leak resources.
3. Two-way clipboard. Text copied on the user's computer can be pasted into the device, and text copied on the device can be pasted on the user's computer.
4. Restricted access. A session is limited to one app of your choice and a defined set of actions. Tell us which app you chose and why. The user must not be able to leave the app, open other apps, or reach system-level controls. Decide which actions to block and justify the list. Enforcement must not rely only on the browser, since a user can tamper with client-side code.
5. Session recording. Each session is recorded automatically. The recording can be played back or downloaded afterwards, and each recording is tied to its session.

Scope

- Scaling is not required. Supporting 2 to 3 simultaneous instances on one machine is enough.
- Do not build autoscaling or clustering. The deployment itself should still work reliably on a real server.
- A simple design that works reliably is preferred over a complex one that is half finished.
- The core requirements plus one or two bonuses make a strong submission. We do not expect all five.

Deployment

Submit a deployed link that we can open in a browser and test without installing anything. You may use any cloud provider or hosting service to run it, free or paid. The only rule is that the hosting provides generic infrastructure (virtual machines, containers, storage, networking). It must not provide the Android device streaming itself.

The backend must run on a server, not on your own computer, so that we can test it at any time without depending on your machine being on. Getting this running and keeping it running (server access, networking, process management, cleaning up after sessions) is part of the challenge, and we want to see how you handle it. Keep it available while we evaluate your submission. State in the README where it is hosted and any limits we should know about, such as the number of simultaneous sessions.

Using AI

We are not against using AI. We encourage it, and we expect you to use it. What we want to understand is how you use it: what you ask, what you decide yourself, what you check, and what you change when the output is wrong. Pasting from an AI chat still means deciding what to ask, what to keep, and where to put it, and that judgement is part of what we assess.

If you use an AI coding agent (any tool that reads an instruction file such as AGENTS.md or CLAUDE.md), the following is compulsory. Paste the instruction below into that file before you start, and commit the resulting PROCESS_LOG.md to your repository as you work, not at the end.

```
Maintain a file called PROCESS_LOG.md in the project root throughout this work. After each meaningful step, append an entry with: the time, the user's exact prompt (verbatim, not summarised), what you did in response, any errors or failures you hit, and what the user decided next. Record dead ends and abandoned approaches as well as successes. Never rewrite or delete earlier entries. Keep the file up to date as you go, not at the end.
```

If you use an AI chat instead (in a browser or an app), share a public link to every conversation you used, so we can read the full exchange. If your tool cannot produce a public link, export the conversation and add it to the repository.

In either case, add a short section to your write-up, in your own words, covering the main decisions you made that the AI did not suggest, and at least one place where the AI was wrong or unhelpful and how you noticed. Please do not edit or clean up the logs or chats. A messy record is fine. Using AI will not count against you. A missing or edited record will.

Deliverables

1. A publicly accessible Git repository containing all of the backend and frontend code, with everything needed to run the project.
2. The deployed link, with any credentials or steps needed to try each feature.
3. A live demo video of 3 to 5 minutes. Record the deployed version in use, not a mock-up. Show the device responding in real time, walk through each feature you built, and narrate what you are doing. It should be one continuous recording, without cuts, so we can see the real behaviour.
4. A README with local setup steps and instructions to test each feature.
5. An architecture write-up of one to two pages covering how the screen reaches the browser, how input reaches the device, and how isolation and restriction are enforced. Include the alternatives you considered and why you rejected them.
6. A short "What went wrong" section listing the dead ends and problems you hit, and how you got past them.
7. A "With more time" section covering how your solution would scale beyond a few users and the main security risks you see.
8. Your AI record: the PROCESS_LOG.md file from your AI agent (compulsory if you used one), or public links to your AI chat conversations. Commit the log to the repository or link everything from your README (see Using AI).
9. A short section in your write-up, in your own words, on the decisions you made that the AI did not suggest and where the AI was wrong (see Using AI).

Evaluation criteria

Area	Weight	What we look at
Core functionality	30%	Stream works, input is accurate, behaviour is stable on the deployed link
Bonus features	25%	Depth and correctness of each one attempted
Problem solving and use of AI	25%	How you researched, experimented, and recovered from failures, and how you directed and checked your AI tools, as shown in the AI record, the write-up and the "What went wrong" section
Engineering quality	10%	Code structure, error handling, cleanup of unused instances

Area	Weight	What we look at
Communication	10%	Clarity of the write-up and demo, and awareness of trade-offs such as latency versus quality and strength of isolation

Deadline, questions and a note

You have 72 hours from the moment you receive this assignment. Submit everything before the deadline, and tell us how much time you actually spent.

If anything is unclear, reply to the email you received this assignment from, or write to rohit.ranjan@myhealthtick.com.

You are not expected to know Android streaming, WebRTC, or any particular technology before starting this assignment. We are evaluating how you learn and solve the problem, not prior familiarity with the specific stack.